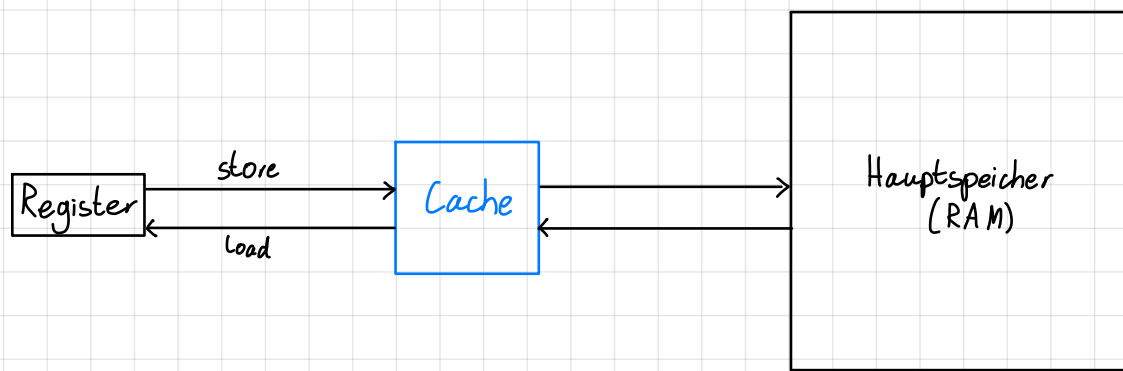


# ERA Tutorium 5

|                  |                    |              |                              |
|------------------|--------------------|--------------|------------------------------|
| Leopold<br>Cario | <u>Termin:</u>     | <u>Raum:</u> | <u>Zulip:</u>                |
|                  | · Mittwoch 10:00   | 03.13.010    | ERA Tutorium - Mi - 1000 - 1 |
|                  | · Donnerstag 12:00 | 00.08.059    | ERA Tutorium - Do - 1200 - 1 |



Räumliche Lokalität z.B. Loops über Arrays

Zeitliche Lokalität: z.B. Variable: Deklaration, Operationen

$$\text{Average Memory Access Time} = \text{Hit Rate} \cdot \text{Hit Latency} + \text{Miss Rate} \cdot \text{Miss Latency}$$

Cache:

| Zeile | Tag | Daten |
|-------|-----|-------|
| 0     |     |       |
| 1     |     |       |
| 2     |     |       |
| 3     |     |       |
| 4     |     |       |
| 5     |     |       |
| 6     |     |       |
| 7     |     |       |

Cachezeilenlänge

Set 0 (Zeilen 0-3)  
Set 1 (Zeilen 4-7)

n-fach assoziativ

- = n Zeilen pro Set
- = jede Adresse kann in n verschiedene Zeilen.

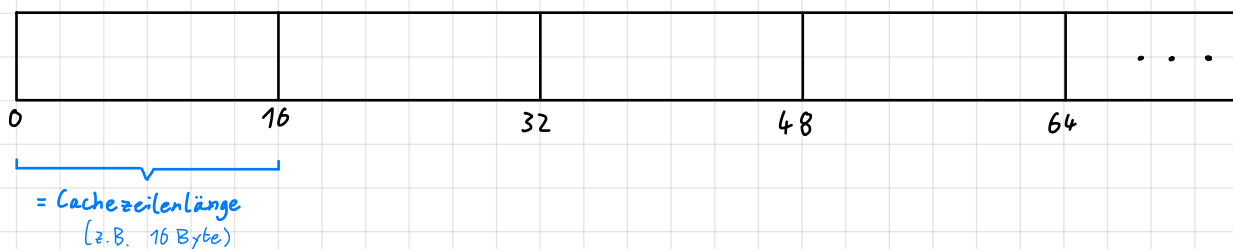
Direct-Mapped

- = 1-fach assoziativ
- = jede Adresse kann in eine Zeile

Vollassoziativ

- = 'einzelnes großes Set'
- = jede Adresse kann in jede Zeile

Hauptspeicher:



Der ganze Block wird in die Cachezeile geladen  $\rightarrow$  räumliche Lokalität

Um trotzdem jedes Byte einzeln adressieren zu können, gibt es Offset-Bits.

Speicheradresse:

0x34D9 = 0b 0011 0100 1101 1001  
0011 0100 1101 1001

$$\# \text{Offsetbits} = \lceil \log_2(\text{Cachezeilenlänge}) \rceil$$

$$(\lceil \log_2(16) \rceil = 4)$$

$$\# \text{Indexbits} = \lceil \log_2(\# \text{Cachesets}) \rceil$$

$$(\lceil \log_2(2) \rceil = 1)$$

$$\# \text{Tagbits} = \text{Adresslänge} - \# \text{Offsetbits} - \# \text{Indexbits}$$

Lehrstuhl für  
Rechnerarchitektur & Parallele Systeme  
Prof. Dr. Martin Schulz  
Dominic Prinz  
Jakob Schäffeler

Lehrstuhl für  
Design Automation  
Prof. Dr.-Ing. Robert Wille  
Stefan Engels  
Benjamin Hien

# Einführung in die Rechnerarchitektur

Wintersemester 2025/2026

## Übungsblatt 5: Caching

17.11.2025 – 21.11.2025

### 1 Cachestruktur und -aufbau

Für die folgenden Fragen werde ein System betrachtet, das nur einen Rechenkern mit einem Cache besitzt. Dieser Cache habe 8 Cachezeilen und eine Cachezeilenlänge von 32 Byte. Die Architektur sei eine reine 32-Bit Architektur und alle Zugriffe 32 Bit breit. Der Speicher sei wie üblich Byte-adressierbar. Vergleichen Sie in den folgenden Teilaufgaben die verschiedenen Cache-Typen, indem Sie die Fragen jeweils für einen Direct-Mapped Cache, einen 2-fach assoziativen Cache und einen voll-assoziativen Cache beantworten.

- a) Wie viele Cachezeilenmengen (Cache-Sets) hat der Cache?

Direct-Mapped: 8

2-fach assoziativ:  $8:2 = 4$

Vollassoziativ: 1

|   |   |   |   |
|---|---|---|---|
| 0 | x | x | x |
| 1 | x | x | x |
| 2 | x | x | x |
| 3 | x | x | x |
| 4 | x | x | x |
| 5 | x | x | x |
| 6 | x | x | x |
| 7 | x | x | x |

- b) Wie viele Bits benötigt man für Offset, Index und Tag?

|                   | Offset | Index | Tag |
|-------------------|--------|-------|-----|
| Direct-Mapped     | 5      | 3     | 24  |
| 2-fach assoziativ | 5      | 2     | 25  |
| Vollassoziativ    | 5      | 0     | 27  |

### 2 Hit or Miss?

Gegeben sei folgender Ausschnitt eines C-Programms:

```

int data[1000];
int sum = 0;

// ABSCHNITT 1
for (int i = 0; i < 1000; i += 64) {
    sum = sum + data[i];
}

```

$$\left\lceil \frac{1000}{64} \right\rceil = 16$$

|               | Tag         | Index | Offset |
|---------------|-------------|-------|--------|
| 1. data[0]:   | 0000 0000 0 | 00    | 0 0000 |
| 2. data[64]:  | 0000 1001 0 | 00    | 0 0000 |
| 3. data[128]: | 0000 0010 0 | 00    | 0 0000 |
| 4. data[192]: | 0000 0011 0 | 00    | 0 0000 |

|                            | Tag         | Index | Offset |
|----------------------------|-------------|-------|--------|
| 1. <code>data[0]</code> :  | 0000 0000 0 | 00    | 0 0000 |
| 2. <code>data[32]</code> : | 0000 0000 1 | 00    | 0 0000 |
| 3. <code>data[64]</code> : | 0000 000 10 | 00    | 0 0000 |
| 4. <code>data[96]</code> : | 0000 000 11 | 00    | 0 0000 |

```

}
// ABSCHNITT 2
for (int i = 0; i < 1000; i += 32) {
    sum = sum + data[i];
}

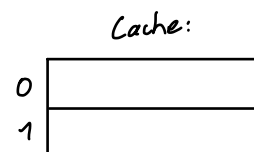
```

$\frac{1000}{32} = 32$

Verwenden Sie für die folgenden Aufgaben den 2-fach assoziativen Cache aus Aufgabe 1. Nehmen Sie an, dass nur das `data` Array im Speicher liegt und alle anderen Werte in Registern gehalten werden. Der Cache sei zu Beginn leer. Gehen Sie weiter davon aus, dass ein `int` 32-Bit groß ist und stets vollständig in einer Cachezeile liegt. Alle Speicherzugriffe erfüllen das jeweils erforderliche Alignment.

a) Wie viele „Capacity Misses“ sind in den beiden Abschnitten zu erwarten?

1. 0



2. 16

0, 32, 64, 96, ...

b) Wie viele „Cold Misses“ sind zu erwarten?

Ansatz: Betrachten Sie die Adressen der einzelnen Array-Elemente.

1. 16

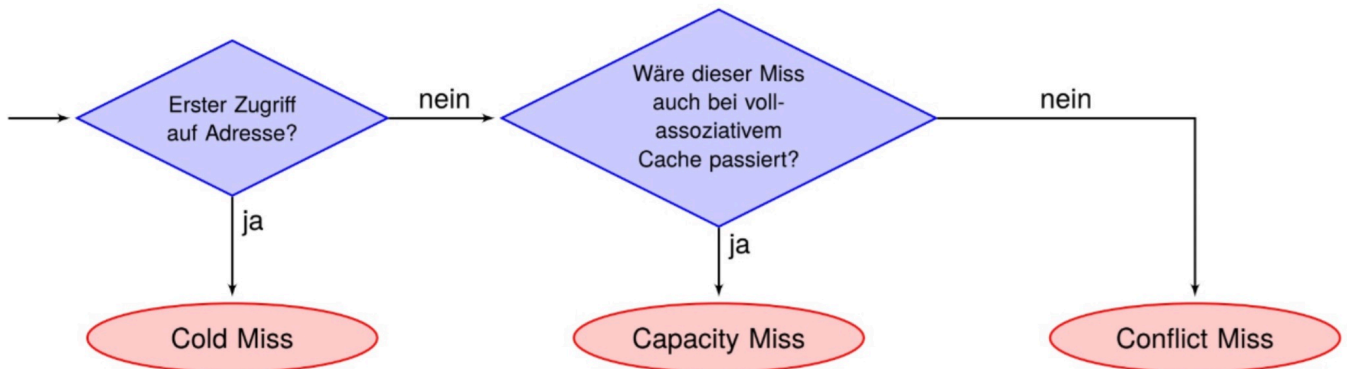
2. 16

c) Knobelaufgabe: Wie ändern sich diese Werte, wenn die Variablen `sum` und `i` nicht in Registern sondern direkt nach dem Array `data` befinden?

## Arten von Misses

- ▶ Cold Miss:
  - Die Daten an dieser Adresse werden zum ersten Mal in den Cache geladen.
- ▶ Conflict Miss:
  - Die Daten an dieser Adresse waren bereits im Cache, wurden aber verdrängt, obwohl noch Platz im Cache war.
- ▶ Capacity Miss:
  - Die Daten an dieser Adresse waren bereits im Cache, wurden aber verdrängt. Dies wäre selbst bei einem vollassoziativen Cache geschehen.

## Arten von Misses



- Random
- LRU = Last recently used
- LFU = Least frequently used
- FIFO = First in, first out

### 3 Ersetzungsstrategien

Gegeben sei ein 4-fach assoziativer Cache mit 16 Cachezeilen. Pro Cachezeile werden 4 Byte abgespeichert. In den folgenden Tabellen ist jeweils das Cache-Set 0 angegeben. Cachen sie die folgenden Werte jeweils einmal mit LFU und LRU als Ersetzungsstrategie und füllen sie in den Tabellen aus. Als Vereinfachung wird von jeder Adresse nur der Tag angegeben und Index und Offset werden in dieser Aufgabe ignoriert. Falls die Ersetzungsstrategie zu keinem eindeutigen Ergebnis führt ersetzen Sie immer die niederwertigere Cachezeile mit dem neuen Tag.

- a) Tragen Sie in die Tabelle die für die Blöcke gespeicherten Tags mit LRU als Ersetzungsstrategie ein.

*Beispiel: In Zeitschritt 1 wird die Adresse mit dem Tag 12 gecached. Da das Datum an der Adresse noch nicht im Cache vorhanden ist handelt es sich um ein Miss. Der Wert wird in Cachezeile 1 und der letzte Access in **Last Use** von Cachezeile 1 gespeichert.*

| Time | Tag | Line 0 |    | Line 1 |    | Line 2 |    | Line 3 |    | Hit/Miss |
|------|-----|--------|----|--------|----|--------|----|--------|----|----------|
|      |     | Tag    | LU | Tag    | LU | Tag    | LU | Tag    | LU |          |
| 1    | 12  | 12     | 1  | –      | –  | –      | –  | –      | –  | Miss     |
| 2    | 20  | 12     | 1  | 20     | 2  | –      | –  | –      | –  | Miss     |
| 3    | 12  | 12     | 3  | 20     | 2  | –      | –  | –      | –  | Hit      |
| 4    | 12  | 12     | 4  | 20     | 2  | –      | –  | –      | –  | Hit      |
| 5    | 28  | 12     | 4  | 20     | 2  | 28     | 5  | –      | –  | Miss     |
| 6    | 32  | 12     | 4  | 20     | 2  | 28     | 5  | 32     | 6  | Miss     |
| 7    | 38  | 12     | 4  | 38     | 7  | 28     | 5  | 32     | 6  | Miss     |
| 8    | 25  | 25     | 8  | 38     | 7  | 28     | 5  | 32     | 6  | Miss     |
| 9    | 12  | 25     | 8  | 38     | 7  | 12     | 9  | 32     | 6  | Miss     |
| 10   | 32  | 25     | 8  | 38     | 7  | 12     | 9  | 32     | 10 | Hit      |

- b) Tragen Sie in die Tabelle die für die Blöcke gespeicherten Tags mit LFU als Ersetzungsstrategie ein.

Beispiel: In Zeitschritt 1 wird die Adresse mit dem Tag 12 gecached. Da das Datum an der Adresse noch nicht im Cache vorhanden ist handelt es sich um ein Miss. Der Wert wird in Cachezeile 1 gespeichert und der Counter auf 1 gesetzt.

| Time | Tag | Line 0 |       | Line 1 |       | Line 2 |       | Line 3 |       | Hit/Miss |
|------|-----|--------|-------|--------|-------|--------|-------|--------|-------|----------|
|      |     | Tag    | Count | Tag    | Count | Tag    | Count | Tag    | Count |          |
| 1    | 12  | 12     | 1     | –      | –     | –      | –     | –      | –     | Miss     |
| 2    | 20  | 12     | 1     | 20     | 1     |        |       |        |       | Miss     |
| 3    | 12  | 12     | 2     | 20     | 1     |        |       |        |       | Hit      |
| 4    | 12  | 12     | 3     | 20     | 1     |        |       |        |       | Hit      |
| 5    | 28  | 12     | 3     | 20     | 1     | 28     | 1     |        |       | Miss     |
| 6    | 32  | 12     | 3     | 20     | 1     | 28     | 1     | 32     | 1     | Miss     |
| 7    | 38  | 12     | 3     | 38     | 1     | 28     | 1     | 32     | 1     | Miss     |
| 8    | 25  | 12     | 3     | 25     | 1     | 28     | 1     | 32     | 1     | Miss     |
| 9    | 12  | 12     | 4     | 25     | 1     | 28     | 1     | 32     | 1     | Hit      |
| 10   | 32  | 12     | 4     | 25     | 1     | 28     | 1     | 32     | 2     | Hit      |

c) Vergleichen Sie die Ersetzungsstrategien. Was sind Vor- und Nachteile der jeweiligen Strategien?

- LFU: Gut bei repetitiven Zugriffsmustern, relativ einfach zu implementieren; kann schlecht bei unregelmäßigen Zugriffsmustern sein
- LRU: Gute Leistung bei Workloads mit temporärer Lokalität; passt sich gut an ändernde Workloads an; schwierig zu implementieren; schlecht bei zufälligen oder verstreuten Zugriffen

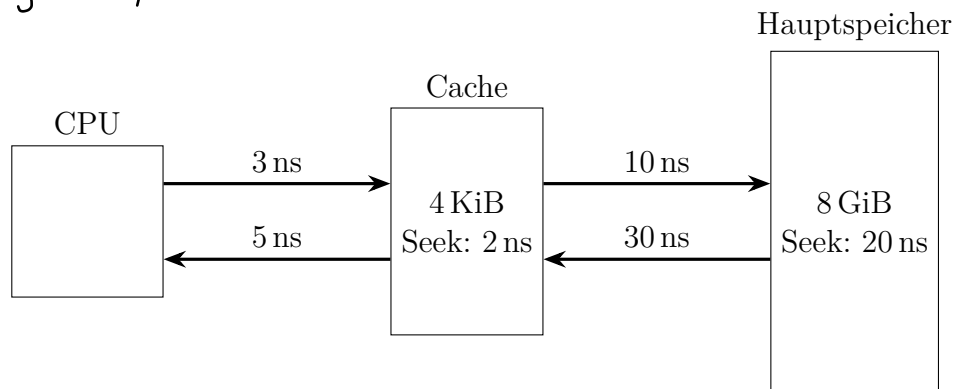
## 4 Speicherzugriffszeit

Der Speicher eines Computers sei wie folgt aufgebaut:



$$\text{Miss Penalty} = \text{Miss Latency} - \text{Hit Latency}$$

$$\text{Average Memory Access Time} = \text{Hit-Rate} \cdot \text{Hit-Latency} + \text{Miss-Rate} \cdot \text{Miss-Latency}$$



- a) Es sei eine Average Memory Access Time von 15 ns gegeben. Berechnen Sie Hit-Latency, Miss-Penalty, Miss-Latency und Miss-Rate.

$$\begin{aligned} \text{Hit-Latency: } & 3 \text{ ns} + 2 \text{ ns} + 5 \text{ ns} = 10 \text{ ns} \\ \text{Miss-Penalty: } & 10 \text{ ns} + 20 \text{ ns} + 30 \text{ ns} = 60 \text{ ns} \\ \text{Miss-Latency: } & \text{Hit-Latency} + \text{Miss Penalty} = 70 \text{ ns} \end{aligned}$$

- b) Ein:e Programmierer:in optimiert das Programm um die Hit-Rate im Cache zu maximieren. Dadurch sinkt die durchschnittliche Speicherzugriffszeit um  $\frac{1}{3} = 33.33\%$ . Dennoch erhöht sich die Ausführungszeit. Woran könnte das liegen?
- c) Was sind Gründe, weshalb sich die Laufzeiten eines Programms heutzutage nicht mehr so einfach wie oben berechnen lassen?

Es gibt diverse Gründe, wieso Laufzeiten nicht mehr vorhersehbar sind. Hier eine Auswahl:

- Moderne Prozessoren sind in der Regel out-of-order Prozessoren. Diese können Instruktionen umsortieren, damit Speicherzugriffe sich überschneiden können. Damit kann die Anzahl an Wartezyklen (memory stall cycles) teils deutlich reduziert werden.
- Auf Multikernrechnern gibt es in der Regel mehrere Cache-Ebenen, die privat oder geteilt sein können und über Coherence-Protokolle verfügen. Hit/Miss-Rates sowie die Latenzen können somit von anderen Prozessorkernen beeinflusst werden.
- Heutzutage laufen Programme oft nicht mehr alleine auf einem System. Interferenzen durch das Betriebssystem oder andere Programme können starke Auswirkungen auf den Cache und die Laufzeit haben.

$$a) \text{ AMAT} = \text{HitRate} \cdot \text{Hit Latency} + \text{Miss Rate} \cdot \text{Miss Latency}$$

$$15 \text{ ns} = \text{HitRate} \cdot 10 \text{ ns} + \text{Miss Rate} \cdot 70 \text{ ns}$$

$$15 \text{ ns} = (1 - \text{MissRate}) \cdot 10 \text{ ns} + \text{Miss Rate} \cdot 70 \text{ ns}$$

$$15 \text{ ns} = 10 \text{ ns} - \text{MissRate} \cdot 10 \text{ ns} + \text{Miss Rate} \cdot 70 \text{ ns}$$

$$15 \text{ ns} = 10 \text{ ns} + \text{MissRate} \cdot 60 \text{ ns} \quad | - 10 \text{ ns}$$

$$5 \text{ ns} = 60 \text{ ns} \cdot \text{Miss Rate} \quad | : 60 \text{ ns}$$

$$\underline{\text{Miss Rate} = \frac{5}{60} = \frac{1}{12} \approx 8,3\%}$$